

```

#!/usr/bin/env python3
import os
import tempfile
import shutil
from pathlib import Path

import yt_dlp
from telegram import Update, Bot, InputMediaPhoto, InputMediaVideo
from telegram.ext import Application, CommandHandler, MessageHandler, filters, Co

BOT_TOKEN = os.environ["BOT_TOKEN"]
CHANNEL = "@sohrani_obsudim"

WAITING_LINK, WAITING_CAPTION = range(2)
user_links = {}

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "👋 Привет! Я постю контент из Instagram в канал @sohrani_obsudim.\n\n"
        "Просто отправь мне ссылку на Instagram пост 📌"
    )

async def handle_link(update: Update, context: ContextTypes.DEFAULT_TYPE):
    url = update.message.text.strip()
    if "instagram.com" not in url:
        await update.message.reply_text("❌ Это не Instagram ссылка. Попробуй ещё")
        return WAITING_LINK
    user_links[update.effective_user.id] = url
    await update.message.reply_text("✎ Напиши подпись для поста (или `` чтобы ё")
    return WAITING_CAPTION

async def handle_caption(update: Update, context: ContextTypes.DEFAULT_TYPE):
    caption = update.message.text.strip()
    if caption == "-":
        caption = ""
    user_id = update.effective_user.id
    url = user_links.pop(user_id, None)
    if not url:
        await update.message.reply_text("❌ Начни сначала — отправь ссылку.")
        return ConversationHandler.END

```

```

await update.message.reply_text("⌚ Скачиваю из Instagram...")
tmp_dir = tempfile.mkdtemp(prefix="insta_")
try:
    files = download_media(url, tmp_dir)
    if not files:
        await update.message.reply_text("❌ Не удалось скачать. Пост должен б
        return ConversationHandler.END
    await update.message.reply_text(f"📁 Скачано {len(files)} файл(ов). Постк
    bot = Bot(token=BOT_TOKEN)
    await post_media(bot, files, caption)
    await update.message.reply_text("✅ Готово! Опубликовано в @sohrani_obsuc
except Exception as e:
    await update.message.reply_text(f"❌ Ошибка: {e}")
finally:
    shutil.rmtree(tmp_dir, ignore_errors=True)
return WAITING_LINK

def download_media(url, tmp_dir):
    opts = {
        "outtmpl": os.path.join(tmp_dir, "%(id)s.%(ext)s"),
        "format": "best",
        "quiet": True,
        "no_warnings": True,
    }
    files = []
    with yt_dlp.YoutubeDL(opts) as ydl:
        info = ydl.extract_info(url, download=True)
        entries = info.get("entries") or [info]
        for e in entries:
            if e:
                f = ydl.prepare_filename(e)
                if os.path.exists(f):
                    files.append(f)
    return files

def is_video(f):
    return Path(f).suffix.lower() in {".mp4", ".mov", ".avi", ".mkv", ".webm"}

async def post_media(bot, files, caption):
    if len(files) == 1:
        f = files[0]
        with open(f, "rb") as fh:
            if is_video(f):
                await bot.send_video(chat_id=CHANNEL, video=fh, caption=caption,

```

```

        else:
            await bot.send_photo(chat_id=CHANNEL, photo=fh, caption=caption)
    else:
        media, handles = [], []
        for i, f in enumerate(files[:10]):
            fh = open(f, "rb")
            handles.append(fh)
            cap = caption if i == 0 else None
            media.append(InputMediaVideo(fh, caption=cap) if is_video(f) else InputMediaPhoto(fh, caption=cap))
        try:
            await bot.send_media_group(chat_id=CHANNEL, media=media)
        finally:
            for fh in handles:
                fh.close()

def main():
    app = Application.builder().token(BOT_TOKEN).build()
    conv = ConversationHandler(
        entry_points=[MessageHandler(filters.TEXT & ~filters.COMMAND, handle_link)],
        states={
            WAITING_LINK: [MessageHandler(filters.TEXT & ~filters.COMMAND, handle_link)],
            WAITING_CAPTION: [MessageHandler(filters.TEXT & ~filters.COMMAND, handle_caption)],
        },
        fallbacks=[CommandHandler("start", start)],
    )
    app.add_handler(CommandHandler("start", start))
    app.add_handler(conv)
    print("🤖 Бот запущен!")
    app.run_polling()

if __name__ == "__main__":
    main()

```